# BookLeaf Publishing

## Frontend & Full-Stack Integration Master Guide

Line-by-Line Code Walkthrough, Integration Protocol, and Interview Q&A

## 1. Full-Stack Integration & Connection Architecture

The application operates as two decoupled environments: the Client Single Page Application (SPA) powered by React 19 + Vite running on port 5173, and the REST API Server powered by Express 5 running on port 8080. They communicate over HTTP/JSON with stateful cookie sessions.

### The Authentication Handshake & Session Lifecycle

• **1. Login Submission:** The author or admin submits credentials via POST /api/auth/login. The payload travels over JSON body { email, password }.

• **2. Cookie Issuance:** Upon bcrypt password verification, Express signs a JWT with user ID and issues an HTTP response header: Set-Cookie: bookleaf=<token>; HttpOnly; SameSite=Lax; Max-Age=86400; Path=/.

• **3. Automatic Cookie Transmission:** Because client/src/api.js sets 'credentials: include', the browser automatically attaches the 'bookleaf' cookie to all subsequent API calls without any manual Authorization headers.

• **4. Session Hydration (/api/auth/me):** On application load, React calls GET /api/auth/me inside an initial useEffect. The backend validates the cookie and returns the user's role and profile. If valid, the user state is populated; if invalid or absent, loading completes and displays the Login screen.

### Cross-Origin Resource Sharing (CORS) Mechanics

• **Why credentials: true matters:** By default, browsers block cross-origin requests from reading cookies or sending them to a different port (5173 -> 8080). Express explicitly sets origin: CLIENT_URL and credentials: true. In response to preflight OPTIONS requests, the server returns Access-Control-Allow-Credentials: true and Access-Control-Allow-Origin: http://localhost:5173.

### Near Real-Time Updates via HTTP Polling

• **Author Portal (5s interval):** MyTickets.jsx executes a polling loop every 5,000ms. If an admin drafts and sends a response, the author sees it update automatically without reloading.

• **Admin Portal (10s interval):** AdminQueue.jsx polls /api/tickets every 10,000ms. New tickets submitted by authors appear in the queue automatically.

• **Memory Leak Prevention:** Both components store the timer ID returned by setInterval and return a cleanup callback: return () => clearInterval(timer). This prevents ghost network calls when navigating away.

## 2. Frontend Design Decisions: Why We Chose This Stack

• **React 19 + Vite vs. Next.js:** Next.js (App Router/SSR) introduces server components, node server runtimes, and hydration mismatches for authenticated internal dashboards. A client-side Vite SPA compiles in <1.5s, provides instant Hot Module Replacement (HMR), produces static assets deployable on any CDN/Vercel, and keeps the code simple and readable.

• **Native React State (useState) vs. Redux / Zustand:** For an application with clean route separation, a global Redux store creates unnecessary boilerplate (actions, reducers, dispatchers). Role-based session state is hoisted in App.jsx and passed to Navbar; page-level data lives cleanly inside each component's local state.

• **Centralized api.js vs. Axios:** Modern browser fetch is native and requires zero external bundle size. Our custom api() wrapper standardizes base URLs, headers, error handling, and 'credentials: include' in just 15 lines of easily explainable code.

• **Vanilla CSS Custom Properties vs. Tailwind / Component Libraries:** Using custom CSS variables (--primary, --accent, --border) keeps the build size under 12KB, eliminates class clutter in JSX, and gives the UI a bespoke BookLeaf publishing brand identity without depending on heavy third-party UI libraries.

## 3. Detailed Line-by-Line Code Walkthrough (Every Frontend File)

## A. Application Entry: client/src/main.jsx

Initializes the React DOM root at document.getElementById('root'). Wraps the application in <StrictMode> (which in development helps detect accidental side effects by double-invoking lifecycles) and <BrowserRouter> (providing the client-side URL history stack).

## B. Root Routing & Auth Orchestrator: client/src/App.jsx

• **const [user, setUser] = useState(null):** Holds the currently authenticated user object (name, email, role, authorId). If null, user is considered logged out.

• **const [loading, setLoading] = useState(true):** Controls initial boot state. While verifying cookie with /api/auth/me, renders a clean loading screen.

• **useEffect with .finally(() => setLoading(false)):** Ensures loading is ALWAYS flipped to false regardless of whether the network succeeded or failed. This eliminates the 'stuck on loading' bug if the backend is temporarily offline.

• **Role-Based Route Branching:** If user.role === 'admin', the router strictly exposes <AdminQueue /> at '/' and <AdminTicket /> at '/admin/tickets/:id'. If user.role === 'author', it strictly exposes <Books /> at '/', <SubmitTicket /> at '/submit', and <MyTickets /> at '/tickets'. Any unrecognized route redirects to '/'.

## C. Fetch Wrapper: client/src/api.js

• **const API = import.meta.env.VITE_API_URL || 'http://localhost:8080/api':** Uses Vite's environment variable syntax to allow dynamic production API routing while defaulting to local port 8080.

• **credentials: 'include':** Mandates that the browser transmits cookies on all cross-origin requests.

• **try...catch block:** Catches network failure exceptions (e.g. server offline) and returns { success: false, message: 'Could not connect to server' } rather than unhandled promise rejections.

## D. Global Navigation: client/src/components/Navbar.jsx

• **useLocation() hook:** Inspects location.pathname to apply the 'active' CSS class to the current navigation link.

• **Role-Aware Links:** Displays Ticket Queue for admins, or My Books / Submit Query / My Tickets for authors.

• **User Avatar & Badge:** Renders user.name.charAt(0) inside an avatar circle, user's name, and their role badge.

• **logout handler:** Calls POST /api/auth/logout (which instructs the backend to clear the cookie) and resets local user state via setUser(null).

## E. Login View: client/src/pages/Login.jsx

• **Controlled Inputs:** Maintains email and password state, updated via onChange={(e) => set...(e.target.value)}.

• **handleSubmit (e.preventDefault):** Prevents native HTML form reload, calls api('/auth/login', 'POST', { email, password }), and upon success calls setUser(data.user). If invalid, populates error message.

## F. Author Books Dashboard: client/src/pages/Books.jsx

• **Catalogue Summary:** Renders author's published titles in a responsive grid.

• **Edge Case Handling (Books in Production):** Evaluates b.publication_date. If published, renders a 4-column financial grid (Copies Sold, Royalty Earned, Royalty Paid, Royalty Pending). If in production (like BK013 Midnight in Mysore), displays an 'In Production' badge and clarifies that royalty figures appear once live, preventing NaN display.

## G. Ticket Creation Form: client/src/pages/SubmitTicket.jsx

• **Dynamic Book Dropdown:** Fetches author's books on mount and maps them into <option> elements, defaulting to 'General / Account Level' (value='').

• **Submitting State:** Sets submitting = true during the POST call, disabling the button and displaying 'Submitting...' to prevent accidental duplicate submissions.

• **useNavigate('/tickets'):** Upon successful ticket creation, smoothly navigates the author to their ticket list.

## H. Author Tickets View: client/src/pages/MyTickets.jsx

• **5-Second Polling Loop:** Calls api('/tickets/mine') every 5 seconds. Replaces state when new replies arrive.

• **Status Badging:** statusClass(t.status) applies semantic color classes (status-Open, status-In-Progress, status-Resolved).

• **Conversation Thread:** Renders admin responses inside .reply bubbles with whiteSpace: pre-wrap and formatted timestamps.

### I. Admin Ticket Queue: client/src/pages/AdminQueue.jsx

• **Filter Bar:** Controlled dropdowns for Status, Category, Priority, and Date. Appends non-empty filters into URLSearchParams.

• **10-Second Auto-Refresh:** Refetches filtered tickets every 10 seconds.

• **Age Calculation:** ageInDays calculates age from createdAt timestamp in days, helping reps spot aging queries.

• **Priority Highlighting:** Tags Critical (red), High (orange), Medium (yellow), Low (green).

### J. Admin Ticket Management & AI Detail: client/src/pages/AdminTicket.jsx

• **useParams():** Extracts ticket MongoDB _id from URL path.

• **In-Place Updates (update):** Sends PATCH /api/tickets/:id on status, category, or priority changes, or when clicking 'Assign to me'. Shows instant feedback.

• **AI Draft Generation (getDraft):** Calls POST /api/tickets/:id/draft. If regenerate=true, requests a fresh completion. Puts draft text directly into reply textarea for editing.

• **Send Reply (sendReply):** Disables button with 'Sending Reply...'. Posts to /api/tickets/:id/reply. On success, updates ticket responses, clears textarea, and triggers a green confirmation banner ('  Reply sent to author successfully!') that auto-dismisses after 4 seconds.

• **Internal Notes (addNote):** Adds private notes visible strictly to administrators.

## 4. Full-Stack Interview Questions & Senior Answers

**Q: How does client/src/api.js work, and why did you choose it over Axios?**

Answer: api.js is a lightweight wrapper around the standard Fetch API. It handles URL resolution via import.meta.env, attaches 'Content-Type: application/json', mandates 'credentials: include' so HTTP-only session cookies are sent automatically, and wraps calls in a try/catch block to return safe error objects if the server is unreachable. We avoided Axios because native Fetch is built into every modern browser, adding zero kilobytes to our JavaScript bundle while fulfilling all our requirements.

**Q: Why did you use polling in useEffect and how did you prevent memory leaks and zombie network calls?**

Answer: We used setInterval inside useEffect to poll endpoints (5s for authors, 10s for admins) to deliver near-real-time updates without page refreshes. Crucially, we prevent memory leaks by returning a cleanup function from the useEffect hook: return () => clearInterval(timer). When the user navigates to another page, React unmounts the component and cancels the interval, guaranteeing no background network calls persist.

**Q: Why was the screen previously stuck on 'Loading...', and how did your fix resolve it?**

Answer: Initially, App.jsx called api('/auth/me').then(...) without a .catch() or .finally(). When the backend server failed to connect to MongoDB, the fetch promise rejected with a network error. Because the rejection was unhandled, setLoading(false) was never called, keeping loading=true permanently. We fixed this by wrapping fetch in a try/catch inside api.js and attaching .finally(() => setLoading(false)) in App.jsx, ensuring the login screen always renders cleanly even if the backend is down.

**Q: How does role-based routing work in App.jsx to prevent authors from accessing admin pages?**

Answer: In App.jsx, we conditionally render distinct <Route> trees based on user.role === 'admin'. If an author is logged in, the admin routes (/admin/tickets/:id) do not even exist in the React Router route tree. If an author manually types an admin URL into their browser, the wildcard <Route path='*' element={<Navigate to='/' />} /> immediately redirects them to their home page. Furthermore, our backend API strictly guards all admin endpoints with adminsOnly middleware.

**Q: Why did you use controlled components for all form inputs instead of uncontrolled refs?**

Answer: Controlled components bind input values directly to React state via value={state} and onChange={(e) => setState(e.target.value)}. This enables instant validation, character length tracking, conditional submit button disabling (e.g. disabled={!reply.trim()}), and programmatic clearing of fields upon successful submission (e.g. setReply('')).

**Q: How did you resolve the issue where clicking 'Send Reply to Author' showed 'Reply sent' in red like an error?**

Answer: Originally, AdminTicket.jsx shared a single state variable called 'message' that was rendered inside a  element in the Manage card. We refactored this by giving each action its own dedicated state (replySuccess, replyError, manageMsg, noteSuccess). Now, sending a reply sets a green banner (  Reply sent to author successfully! ) right below the button, disables the button during in-flight requests, and

clears itself automatically after 4 seconds.

### Q: What is the difference between process.env in Node.js and import.meta.env in Vite?

Answer: Node.js runs on a server with direct operating system access, reading environment variables from process.env via the dotenv package. Vite, however, is a client-side build tool that runs in the browser where process.env does not exist. Vite uses modern ECMAScript module metadata (import.meta.env) and only exposes variables prefixed with VITE_ (such as VITE_API_URL) to client-side code for security.

### Q: How would you upgrade this system if BookLeaf required instant push notifications instead of polling?

Answer: We would implement Server-Sent Events (SSE) or WebSockets. For this one-way server-to-client push use case, Server-Sent Events (SSE) is the cleanest choice: it runs over standard HTTP, automatically handles reconnections, and works seamlessly through proxies without the duplex overhead of WebSockets. The backend would keep an open res connection per author, and whenever an admin replied, the ticket controller would emit a JSON event directly to the client's EventSource.

### Q: How does the frontend handle edge cases like books with no royalties or books in production?

Answer: In Books.jsx, we evaluate b.publication_date. For published titles, we render the full 4-stat financial grid. For books still in production (such as BK013 and BK015 in the sample data), publication_date and mrp are null. Our code detects this, renders an 'In Production' badge, and displays a friendly notice stating that royalty figures will populate once the book goes live, preventing broken tables or NaN calculations.